

Brac University

Department of Electrical & Electronic Engineering

Semester Spring-25

Course Number: EEE103L

Course Title: Computer Programming Laboratory

Section: 06

Lab Report



Experiment no.

Name of the experiment: Introduction to C programming, I/O, and Datatypes

Prepared by:

Name: **Tanzeel Ahmed** ID: **24321367**

Group Number: 04

Other Group members:

<i>Sl.</i>	<i>ID</i>	<i>Name</i>
1.	24121083	Abontika Das
2.	24121181	Tasnim Sultana
3.	24121249	Faria Rahman Oyshi
4.	24121251	Yeasin Khan Sifat

Lab report of experiment 2

Task 1: First find the value of each of the following arithmetic expressions in C. Then step by step, show in detail, how to hand calculate those values? Simply, you will write your investigation report on how C in CodeBlocks found those answers.

- A) $3 + 4.8 * 2$
- B) $12 \% 16$
- C) $2. * 6 / 5$
- D) $5.2 + 12 / 8$
- E) $4 - (6 + 18.0 / 3) / 3$
- F) $17 - 12 * 4$
- G) $2 * 6 / 5.$
- H) $7 / (4 * 2)$
- I) $2 * 6 / 5$
- J) $19 \% 13$
- K) $36 / 60$

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Task 1: Find the value of each of the following arithmetic expressions\n\n");
```

```
    float A, C, D, E, F, G, H, I, K;
```

```
    int B, J;
```

```
    A = 3 + 4.8 * 2;
```

```
    B = 12 % 16;
```

```
    C = 2. * 6 / 5;
```

```
    D = 5.2 + 12 / 8;
```

```
    E = 4 - (6 + 18.0 / 3) / 3;
```

```
    F = 17 - 12 * 4;
```

```
    G = 2 * 6 / 5.;
```

```
    H = 7 / (4 * 2);
```

```
    I = 2 * 6 / 5;
```

```
    J = 19 % 13;
```

```
    K = 36.0 / 60.0;
```

```
    printf("A) 3 + 4.8 * 2 = %f\n", A);
```

```
    printf("B) 12 %% 16 = %d\n", B);
```

```
    printf("C) 2. * 6 / 5 = %f\n", C);
```

```
    printf("D) 5.2 + 12 / 8 = %f\n", D);
```

```

printf("E) 4 - (6 + 18.0 / 3) / 3 = %f\n", E);
printf("F) 17 - 12 * 4 = %f\n", F);
printf("G) 2 * 6 / 5. = %f\n", G);
printf("H) 7 / (4 * 2) = %f\n", H);
printf("I) 2 * 6 / 5 = %f\n", I);
printf("J) 19 %% 13 = %d\n", J);
printf("K) 36.0 / 60.0 = %f\n", K);
return 0;
}

```

Screenshot of code:

```

C exp 1 2ndv.c X
D: > BracU > 4. Fourth Semester (SP-25) > EE103 > Lab report > Exp 2 > C exp 1 2ndv.c > main()
1  #include <stdio.h>
2
3  int main() {
4
5      printf("Task 1: Find the value of each of the following arithmetic expressions\n\n");
6
7      float A, C, D, E, F, G, H, I, K;
8      int B, J;
9      A = 3 + 4.8 * 2;
10     B = 12 % 16;
11     C = 2. * 6 / 5;
12     D = 5.2 + 12 / 8;
13     E = 4 - (6 + 18.0 / 3) / 3;
14     F = 17 - 12 * 4;
15     G = 2 * 6 / 5.;
16     H = 7 / (4 * 2);
17     I = 2 * 6 / 5;
18     J = 19 % 13;
19     K = 36.0 / 60.0;
20     printf("A) 3 + 4.8 * 2 = %f\n", A);
21     printf("B) 12 %% 16 = %d\n", B);
22     printf("C) 2. * 6 / 5 = %f\n", C);
23     printf("D) 5.2 + 12 / 8 = %f\n", D);
24     printf("E) 4 - (6 + 18.0 / 3) / 3 = %f\n", E);
25     printf("F) 17 - 12 * 4 = %f\n", F);
26     printf("G) 2 * 6 / 5. = %f\n", G);
27     printf("H) 7 / (4 * 2) = %f\n", H);
28     printf("I) 2 * 6 / 5 = %f\n", I);
29     printf("J) 19 %% 13 = %d\n", J);
30     printf("K) 36.0 / 60.0 = %f\n", K);
31     return 0;
32 }

```

Output:

```
"D:\BracU\4. Fourth Semester" x + v
Task 1: Find the value of each of the following arithmetic expressions
A) 3 + 4.8 * 2 = 12.600000
B) 12 % 16 = 12
C) 2. * 6 / 5 = 2.400000
D) 5.2 + 12 / 8 = 6.200000
E) 4 - (6 + 18.0 / 3) / 3 = 0.000000
F) 17 - 12 * 4 = -31.000000
G) 2 * 6 / 5. = 2.400000
H) 7 / (4 * 2) = 0.000000
I) 2 * 6 / 5 = 2.000000
J) 19 % 13 = 6
K) 36.0 / 60.0 = 0.600000

Process returned 0 (0x0)   execution time : 0.075 s
Press any key to continue.
```

Expression A: $3 + 4.8 * 2$

Manual Calculation:

According to operator precedence, multiplication (*) is performed first: $4.8 * 2 = 9.6$
Then, add 3 to 9.6: $3 + 9.6 = 12.6$

Investigation report on how C in CodeBlocks found those answers:

4.8 is a floating-point number, so multiplication ($4.8 * 2$) results in 9.6 (float). When adding 3 (an integer) to 9.6 (a float), C converts 3 to 3.0, ensuring floating-point addition.

Final Result: 12.6 (float).

Expression B: $12 \% 16$

Manual Calculation: The modulus operator % returns the remainder of integer division: $12 \div 16 = 0$ remainder 12.

Investigation report on how C in CodeBlocks found those answers:

The modulus operator % works only with integers, so $12 \% 16$ follows integer modulus rules.

Final Result: 12 (int).

Expression C: $2. * 6 / 5$

Manual Calculation: We have to do multiplication first: $2.0 * 6 = 12.0$
Then, divide 12.0 by 5: $12.0 \div 5 = 2.4$

Investigation report on how C in CodeBlocks found those answers:

2.0 is a float, so the entire operation is done in floating-point mode.

Final Result: 2.4 (float).

Expression D: $5.2 + 12 / 8$

Manual Calculation: Division first: $12 \div 8 = 1$

Then, add 5.2 to 1: $5.2 + 1 = 6.2$

Investigation report on how C in CodeBlocks found those answers:

$12 / 8$ is integer division (1, not 1.5). 5.2 is a float, so 1 is converted to 1.0, and floating-point addition occurs.

Final Result: 6.2 (float).

Expression E: $4 - (6 + 18.0 / 3) / 3$

Manual Calculation: Division first: $18.0 \div 3 = 6.0$

Add 6 and 6.0: $6 + 6.0 = 12.0$

Divide 12.0 by 3: $12.0 \div 3 = 4.0$

Subtract 4.0 from 4: $4 - 4.0 = 0.0$

Investigation report on how C in CodeBlocks found those answers:

Since 18.0 is a float, all calculations inside the parentheses become floating-point.

Final Result: 0.0 (float).

Expression F: $17 - 12 * 4$

Manual Calculation: Multiplication first: $12 * 4 = 48$

Then, subtract 48 from 17: $17 - 48 = -31$

Investigation report on how C in CodeBlocks found those answers:

All numbers are integers, so integer arithmetic is used.

Final Result: -31 (int).

Expression G: $2 * 6 / 5$.

Manual Calculation: Multiplication first: $2 * 6 = 12$

Then, divide 12 by 5.0: $12 \div 5.0 = 2.4$

Investigation report on how C in CodeBlocks found those answers:

5.0 is a float, so the entire operation is done in floating-point mode.

Final Result: 2.4 (float).

Expression H: $7 / (4 * 2)$

Manual Calculation: Multiplication inside the parentheses: $4 * 2 = 8$
Then, divide 7 by 8: $7 \div 8 = 0$ (integer division truncates decimal part)

Investigation report on how C in CodeBlocks found those answers:
Since 7 and 8 are integers, integer division applies.

Final Result: 0 (int).

Expression I: $2 * 6 / 5$

Manual Calculation: Multiplication first: $2 * 6 = 12$
Then, divide 12 by 5: $12 \div 5 = 2$

Investigation report on how C in CodeBlocks found those answers:
Since all values are integers, integer division applies.

Final Result: 2 (int).

Expression J: $19 \% 13$

Manual Calculation: The modulus operator gives the remainder of integer division: $19 \div 13 = 1$ remainder 6.

Investigation report on how C in CodeBlocks found those answers:
The modulus operator % works only with integers, so $19 \% 13 = 6$.

Final Result: 6 (int).

Expression K: $36 / 60$

Manual Calculation: Division: $36.0 \div 60.0 = 0.6$

Investigation report on how C in CodeBlocks found those answers:
If the operation involves floats, the result is 0.6. If integers are used, the result is 0 (truncated).

Final Result: 0.6 (Float)

Overall Investigation: In basic mathematics we usually follow the line of parentheses, exponents, multiplication/division and addition/subtraction for hand calculations. In C programming multiplication, division, and modulus have higher precedence than addition and subtraction. This means that in expressions where both types of operators are used, multiplication, division, and modulus are performed first. To make the matter simple the code has been written using the basic structure. First the output has been shown using the

mathematical operations of C and then the breakdown has been done by hand calculation and printed all together.

Task 2: Try the following programs and note what happens. Then try to explain the behaviors line by line.

```
int main()
{
char b;
int i = 257;
double d = 323.142;
b = i;
printf("%c\n",b);
b = (char) i;
printf("%c\n",b);
i = (int) d;
printf("%d\n",i);
b = (char) d;
printf("%c\n",b);
}
```

Code:

```
#include<stdio.h>
int main()
{

printf("Task 2:\n");

char b;
int i = 257;
double d = 323.142;

b = i; //The character b is now defined as integer i
printf("%c\n",b);

b = (char) i; //int to char
printf("%c\n",b); //Trying to print a non-printable character

i = (int) d; //double to int
printf("%d\n",i); //Will printout 323 and remove the decimal points

b = (char) d; //double to char
printf("%c\n",b); //Will printout C because 323.142 converted to 67

}
```

Screenshot of code:

```
C q2.c x
D: > BracU > 4. Fourth Semester (SP-25) > EEE103 > Lab report > Exp 3 > C q2.c > ...
1  #include<stdio.h>
2  int main()
3  {
4
5  printf("Task 2:\n");
6
7  char b;
8  int i = 257;
9  double d = 323.142;
10
11  b = i; //The character b is now defined as integer i
12  printf("%c\n",b);
13
14  b = (char) i; //int to char
15  printf("%c\n",b); //Trying to print a non-printable character
16
17  i = (int) d; //double to int
18  printf("%d\n",i); //Will printout 323 and remove the decimal points
19
20  b = (char) d; //double to char
21  printf("%c\n",b); //Will printout C because 323.142 converted to 67
22
23 }
```

Output:

```
"D:\BracU\4. Fourth Semester x + v - □ x
Task 2:

323
C

Process returned 0 (0x0)   execution time : 0.178 s
Press any key to continue.
|
```

Explanation:

Declaration of variables:

char b; Stores the character of the variable **b**

Int i = 257; Declares the variable integer **i** and stores the numeric value 257

double d = 323.142; Stores the double variable **d** and the value of it is declared as 323.142

```
7 char b;  
8 int i = 257;  
9 double d = 323.142;
```

Implicit casting (b=i)

Since **b** is defined as **char** which can hold 8 bits, so the only lower 8 bits are assigned to **b** which converts **b** into 1 and as a result it becomes a non-printable character which is why there is nothing shown on the output terminal.

```
11 b = i; //The character b is now defined as integer i  
12 printf("%c\n",b);
```

Explicit casting b=(char) i;

Similar to the first implicit casting, only 8 bits of integer **i** are stored in the **char b**. So **b** turns into 1 which results in another non-printable character as there is no visible output.

```
14 b = (char) i; //int to char  
15 printf("%c\n",b); //Trying to print a non-printable character  
16
```

Casting double to int i=(int)d;

According to the question, the **char d=323.142** and storing that as **int** removes the fractions so it converts into 323 as displayed on the output.

```
17 i = (int) d; //double to int  
18 printf("%d\n",i); //Will printout 323 and remove the decimal points  
19
```

Casting double to char b=(char)d;

As displayed in the question, d=323.142, the char converts this numeric value to 8 bits which results in 67 and according to the ASCII system, 67 is defined as character C which is why the output result is C on the terminal.

```
20  b = (char) d; //double to char
21  printf("%c\n",b); //Will printout C because 323.142 converted to 67
22
```

Task 3: Here is a pseudo program that computes the number of miles that light will travel in a specified number of days. Try to compile and run the following C program.

Lightspeed = 186000 days = 1000; //specify number of days here seconds = Days*24*60*60; /*convert to seconds/ distance = lightspeed * Second; //compute distance printf("In " + days + " days light will travel about "); printf(Distance + "miles.");

Correction of the code:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
long long lightspeed = 186000; // The speed of light in miles per second
```

```
int days = 1000; // Specifying the number of days
```

```
long long seconds = days * 24 * 60 * 60; // Conversion days into seconds
```

```
long long distance = lightspeed * seconds; // Calculating the distance of light travelling in the given number of days
```

```
printf("In %d days, light will travel about %lld miles.\n", days, distance); //Printing the result
```

```
return 0;
```

```
}
```

Screenshot of corrected code:

```
q3.c x
D:\BracU\4. Fourth Semester (SP-25) > EEE103 > Lab report > Exp 3 > q3.c > ...
1  #include <stdio.h>
2
3  int main()
4  {
5
6
7  long long lightspeed = 186000; // The speed of light in miles per second
8
9  int days = 1000; // Specifying the number of days
10
11 long long seconds = days * 24 * 60 * 60; // Conversion days into seconds
12
13 long long distance = lightspeed * seconds; // Calculating the distance of light travelling in the given number of days
14
15 printf("In %d days, light will travel about %lld miles.\n", days, distance); //Printing the result
16
17 return 0;
18
19 }
20
```

Output:

```
"D:\BracU\4. Fourth Semester" x + v
In 1000 days, light will travel about 160704000000000 miles.

Process returned 0 (0x0)    execution time : 0.147 s
Press any key to continue.
|
```

Investigation: In the corrected code, long long is used to declare lightspeed because it is a large value. int has been used to declare days as it is not taken as a fraction. For calculating second and distance long long is used again as the calculation is large. To conclude, the printf function was used to show the output and format specifier %d was used for int(days) while %lld was used for distance.

Task 4: Write a program in C that can calculate the area of a circle up to 4 decimal points taking the radius as input. Take the value of radius from the user in the console window and radius can be any integer or decimal fraction number. Print the value of the area.

Code:

```
#include<stdio.h>
```

```
int main()
```

```

{
    printf("Task 04:\n");

    float radius, area, pi=3.1416;

    printf("Please input the radius of the circle: ");
    scanf("%f", &radius);

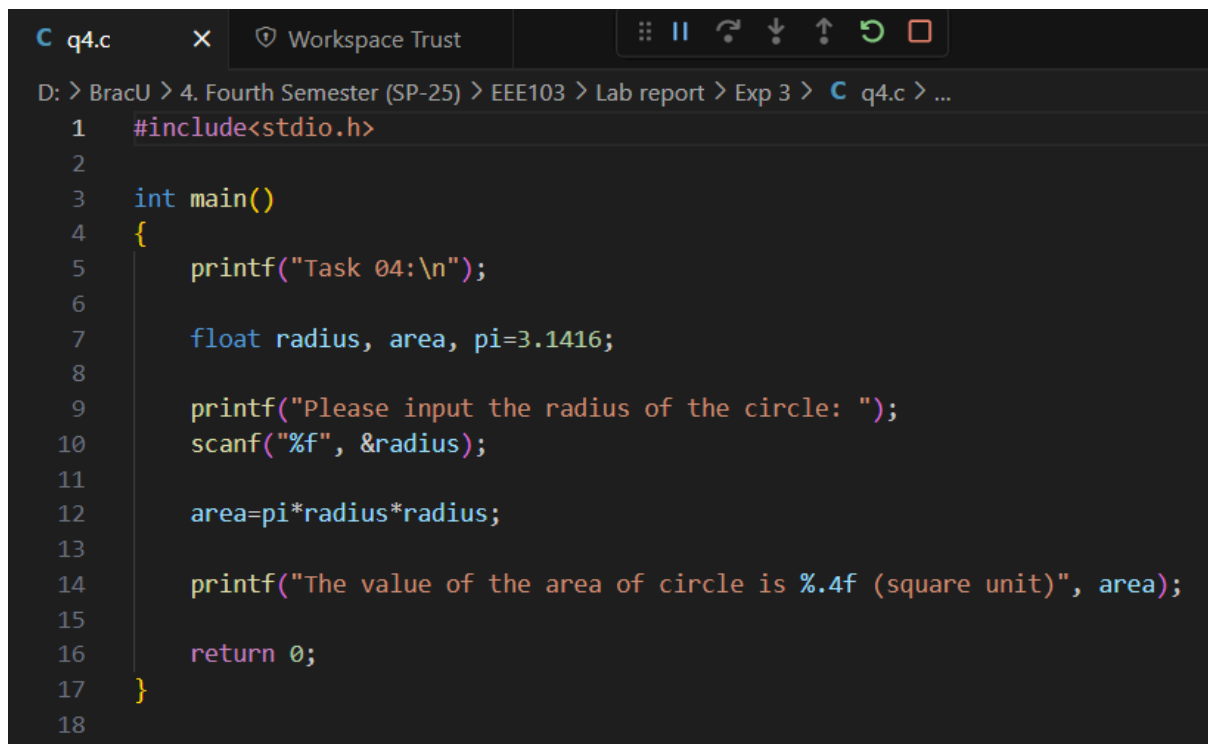
    area=pi*radius*radius;

    printf("The value of the area of circle is %.4f (square unit)", area);

    return 0;
}

```

Screenshot of code:



The screenshot shows a code editor with a dark theme. The file name is 'q4.c'. The path is 'D:\> BracU > 4. Fourth Semester (SP-25) > EEE103 > Lab report > Exp 3 > q4.c > ...'. The code is as follows:

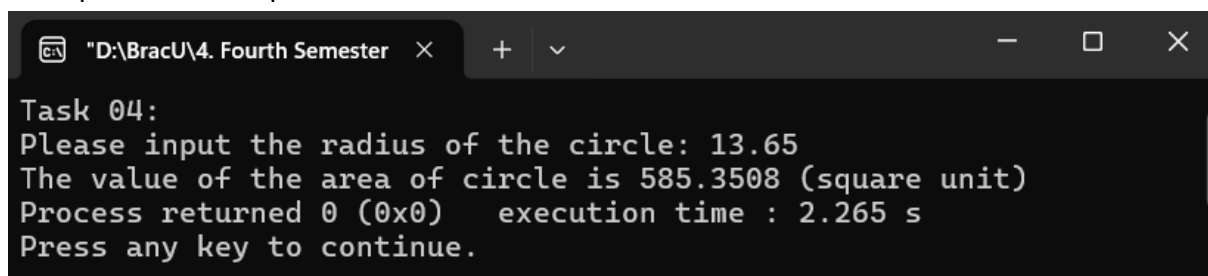
```

1  #include<stdio.h>
2
3  int main()
4  {
5      printf("Task 04:\n");
6
7      float radius, area, pi=3.1416;
8
9      printf("Please input the radius of the circle: ");
10     scanf("%f", &radius);
11
12     area=pi*radius*radius;
13
14     printf("The value of the area of circle is %.4f (square unit)", area);
15
16     return 0;
17 }
18

```

Output:

Example of radius input: 13.65



The screenshot shows a terminal window with the title 'D:\BracU\4. Fourth Semester'. The output is as follows:

```

Task 04:
Please input the radius of the circle: 13.65
The value of the area of circle is 585.3508 (square unit)
Process returned 0 (0x0)   execution time : 2.265 s
Press any key to continue.

```

Investigation: In the code, float has been used to define radius and area assuming there will be fractions in the given number. Also the value of pi is 3.1416 which is also a fraction number. To specify input of radius a prompt has been shown using printf and scanf has been used to take in the number. The mathematical formula has been defined in area and printf has been used to display the area of the circle on the output terminal.